

Coding & Solution

Date	02 October 2026
Team ID	6ab7f31f8c7c66e70672939b
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	[Paste your GitHub / GitLab or hosted link here]
Programming Language(s)	Python, HTML, CSS, JavaScript
Framework(s) Used	FastAPI, Uvicorn, Jinja2, Pydantic; Google Gen AI SDK (Gemini); Hugging Face Transformers + PyTorch (LaMini-Flan-T5)
Key Features Implemented	• Q&A (/qa): concise answer with 3-4 key points, an example and a next step. • Concept explanation (/explain): simple explanation, step-by-step working, example and key points, generated locally by LaMini-Flan-T5 (no API call). • Quiz generator (/quiz): 5 multiple-choice questions (4 options each) with explanations and an interactive scored quiz view. • Summarizer (/summarize): short summary, key points and important terms.

Field	Details
Key Features (continued)	• Learning path (/learn/recommendations): 4-6 staged plan from foundations to advanced, with suggested resources. • Learner levels (beginner / intermediate / advanced) that adapt every response. • FastAPI backend with Pydantic input validation (4000-character limit), JSON cleaning of model output, 45 s timeout, retry and automatic fallback from Gemini 3.5 Flash to Gemini 3.1 Flash-Lite. • Responsive HTML/CSS/JavaScript web UI with copy, regenerate and new-task buttons and a /health status indicator. API key is read from an environment variable only.
Pending / Incomplete Features	• No user accounts or saved history; each request is stateless. • No automated unit/integration tests yet. • Runs locally only; cloud deployment is not done. • Concept explanation uses a small local model, so answers can be shorter and less precise than the Gemini-powered modules.
Setup / Run Instructions	• Install Python 3.10+ and get a Gemini API key from Google AI Studio. • Open a terminal in the EduGenie folder and run: <pre>pip install -r requirements.txt</pre> • Set the key (Windows: set GEMINI_API_KEY=your_key; Mac/Linux: export GEMINI_API_KEY=your_key). • Start the server: <pre>uvicorn main:app --reload</pre> • Open <code>http://127.0.0.1:8000</code> in a browser. • The first Explain request downloads the LaMini-Flan-T5 model, so it may take a little longer.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	

Additional Notes / Comments:

Each feature lives in its own module (`qna.py`, `quiz_module.py`, `summary_module.py`, `learning_path.py`, `explanation_module.py`) and `main.py` routes requests to them.

Error handling covers empty or oversized input, a missing API key, timeouts, busy-server retries with model fallback, malformed AI output and local-model failures, each returning a clear message to the user.

Gemini is used for Q&A, quiz, summary and learning path; the concept explanation module runs locally on LaMini-Flan-T5 (MBZUAI/LaMini-Flan-T5-248M).